



# Introduction to Hippo

Matthew Falcone

# Training objectives

- Understand basics of the implementation of Hippo
  - Mirror mesh, Interaction with OpenFOAM
- Be able to set up cases using Hippo for CHT and thermal-hydraulic coupling
  - Understanding the Hippo's MOOSE input file syntax
  - Understand the new systems such as FoamBCs and FoamVariables

Most importantly:

- You are going to be even bigger fans of Pyvista
  - Trigger warning: there are going to be some crisp visualisations.

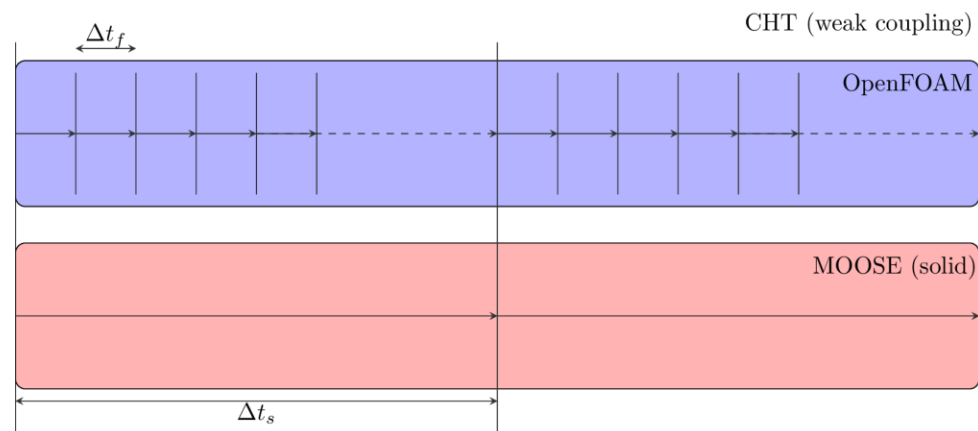
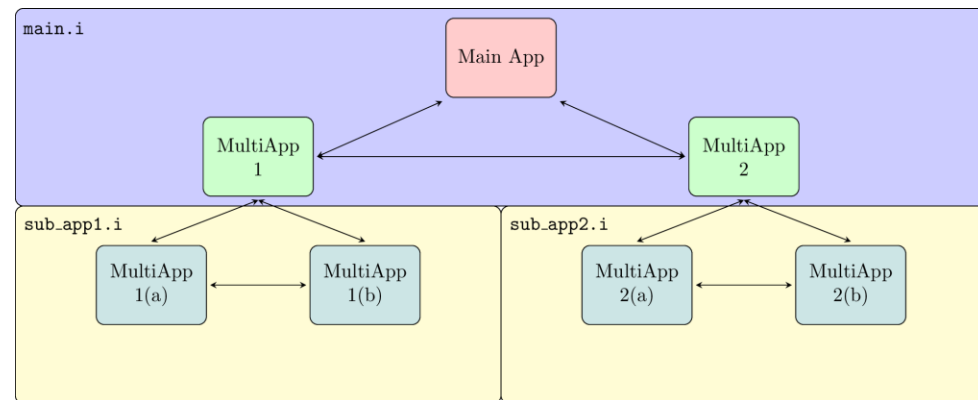
# Motivation and rationale

- At UKAEA, MOOSE is one of the workhorses for our multiphysics simulations
- MOOSE has several features that make it particularly useful for us
  - **Physics modules:** 1D thermal hydraulics, structural mechanics, heat transfer, electromagnetics...
  - **MultiApp system:** extend MOOSE and couple third-party applications into the MOOSE framework.
    - Any MOOSE app can be coupled to any other MOOSE app
    - **Cardinal:** couples OpenMC (neutronics) and nekRS into the MOOSE
    - **TMAP8:** tritium transport code
  - MOOSE has some native CFD capabilities, but these are under-developed and under-used
    - Other tools have much larger user bases and are more established e.g. **OpenFOAM**
- Hippo is a MOOSE app that couples OpenFOAM into the framework
  - So OpenFOAM can be used as an application with MOOSE's MultiApp system



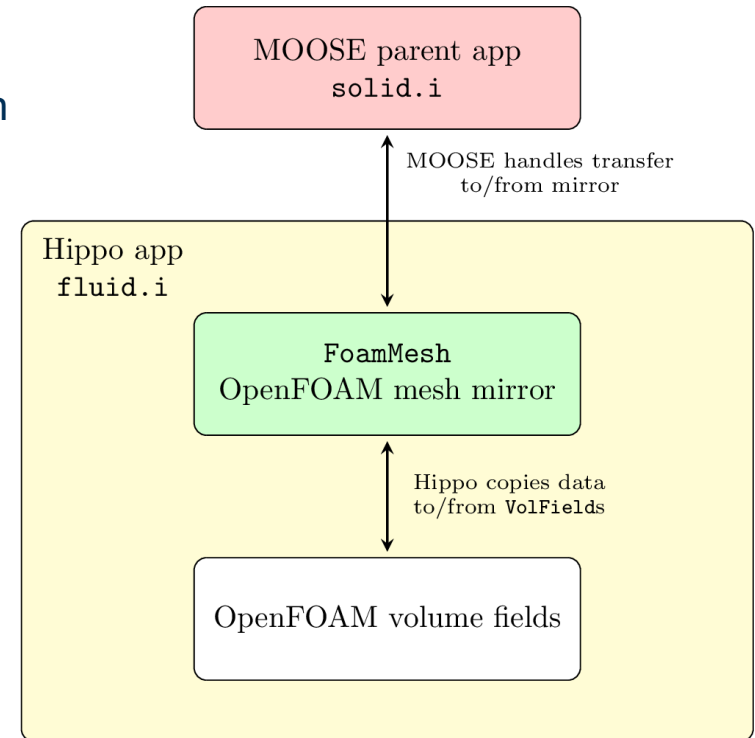
# Hippo in a MOOSE multiphysics simulation

- MOOSE is based on text input files (\*.i)
- Coupling between apps is highly flexible
  - Transfer of data between apps
    - When is data transferred?
    - How to interpolate data?
  - Different time steps for each app
    - Sub-cycling
    - Synchronisation points
  - Fixed-point iteration for tighter convergence
- For Hippo, we define the interface so that these functionalities can be exposed



# Hippo: overview of coupling implementation

- MOOSE is based around the libMesh finite element library
  - The underlying mesh representation is a libMesh mesh
- Hippo uses a libMesh 'mirror' mesh of the OpenFOAM mesh
  - Currently limited to specified boundary patches
  - Only boundary coupling currently implemented
- Data transfer is mediated by the mirror mesh
  - MOOSE transfers data to OpenFOAM mirror mesh
  - Data copied from OpenFOAM mirror to OpenFOAM variables
- Data transfer is possible with MOOSE postprocessors
  - More on this, this afternoon.



# Hippo currently based around the foamRun solver in .org

- FoamRun's main loop directly incorporated into Hippo
  - Spot the difference
- It can use any solver module from OpenFOAM.org
  - We will always use the 'fluid' solver today
- Cannot use standalone solvers unless it is rewritten into the module structure.

```
// Create the outer PIMPLE loop and control structure
pimpleSingleRegionControl pimple(solver.pimple);

// Set the initial time-step
setDeltaT(runTime, solver);

// *****

Info<< nl << "Starting time loop\n" << endl;

while (pimple.run(runTime))
{
    // Update PIMPLE outer-loop parameters if changed
    pimple.read();

    solver.preSolve();

    // Adjust the time-step according to the solver maxDeltaT
    adjustDeltaT(runTime, solver);

    runTime++;

    Info<< "Time = " << runTime.userTimeName() << nl << endl;

    // PIMPLE corrector loop
    while (pimple.loop())
    {
        solver.moveMesh();
        solver.motionCorrector();
        solver.fvModels().correct();
        solver.prePredictor();
        solver.momentumPredictor();
        solver.thermophysicalPredictor();
        solver.pressureCorrector();
        solver.postCorrector();
    }

    solver.postSolve();

    runTime.write();

    Info<< "ExecutionTime = " << runTime.elapsedCpuTime() << "
        << " ClockTime = " << runTime.elapsedClockTime() << "
        << nl << endl;
}

// Create the outer PIMPLE loop and control structure
Foam::pimpleSingleRegionControl pimple(solver.pimple);

// Set the initial time-step
setDeltaT(time, solver);

// ***** //

// Update PIMPLE outer-loop parameters if changed
pimple.read();
solver.preSolve();

// Adjust the time-step according to the solver maxDeltaT
adjustDeltaT(time, solver);
time++;

// TODO: replace std::cout with MOOSE output or a dependency-injected stream.
std::cout << "Time = " << time.userTimeName() << "\n" << std::endl;

// PIMPLE corrector loop
while (pimple.loop())
{
    solver.moveMesh();
    solver.motionCorrector();
    solver.fvModels().correct();
    solver.prePredictor();
    solver.momentumPredictor();
    solver.thermophysicalPredictor();
    solver.pressureCorrector();
    solver.postCorrector();
}

solver.postSolve();

time.write();

std::cout << "ExecutionTime = " << time.elapsedCpuTime() << " s"
    << " ClockTime = " << time.elapsedClockTime() << " s"
    << "\n"
    << std::endl;
```



# Hippo's input file syntax

- Alongside the OpenFOAM case directory, an input file is required that specifies the coupling between MOOSE and OpenFOAM
- Important features
  - FoamMesh: defines OpenFOAM's mirror mesh
  - FoamBCs: block defining how BCs are imposed on OpenFOAM
  - FoamVariables: block defining how variables are transferred to MOOSE
  - FoamTimeStepper: synchronises OpenFOAM and MOOSE's time steps
  - Foam-based postprocessors still defined in the Postprocessor block

```
# Defines the OpenFOAM case location and boundary
# to be mirrored
[Mesh]
  type = FoamMesh
  case = 'foam_cfd'
  foam_patch = interface
  ...
[]

# Defines boundary conditions to be imposed from MOOSE
[FoamBCs]
  ...
[]

# Defines variables to be copied from MOOSE
[FoamVariables]
  ...
[]

[Problem]
  type = FoamProblem
[]

# Controls OpenFOAM's timestepping and synchronisation with MOOSE
[Executioner]
  type = Transient
  ...
  [TimeSteppers]
    [foam]
      type = FoamTimeStepper
    []
  []

# Where Foam-based postprocessors are specified
[Postprocessors]
  ...
[]

# Outputting of mirror mesh and text-based data
[Outputs]
  ...
[]
```

- ```
[FoamBCs]
[temp1]
    type= FoamFixedValueBC
    foam_variable = T
    boundary = 'left right top' # test boundary restrictions
[]
[temp2]
    type= FoamFixedValueBC
    foam_variable = T
    boundary = 'bottom front back' # test boundary restrictions
[]

[AuxKernels]
[T_bc1]
    type= ParsedAux
    variable = temp1
    expression = '0.05 + (x + y + z)*t'
    use_xyz = true
    execute_on= 'TIMESTEP_BEGIN INITIAL'
[]
[T_bc2]
    type= ParsedAux
    variable = temp2
    expression = '0.05 + 2*(x + y + z)*t'
    use_xyz = true
    execute_on= 'TIMESTEP_BEGIN INITIAL'
[]
```



# Boundary conditions types

- Boundary conditions implemented so far
  - FoamFixedValueBC
  - FoamFixedValuePostprocessorBC
  - FoamFixedGradientBC
  - FoamFixedGradientPostprocessorBC
  - FoamDiffusionFluxBC
  - FoamDiffusionFluxPostprocessorBC
  - FoamMassFlowRateInletBC
  - FoamMassFlowRateMappedInletBC
  - FoamScalarBulkMappedInletBC
- We need to make sure the underlying boundary conditions types match
  - fixedValue must be used in 0/T
  - fixedGradient must be used in 0/T

# FoamVariables: copying data from OpenFOAM

- Allows scalar fields from boundary patches to be copied to MOOSE variables
- Like FoamBCs, an AuxVariable is created under-the-hood so it can be transferred
- Applies to
  - Primitive variables such as temperature, T
  - Some function objects such as wallHeatFlux
  - Other custom calculated fields such as heat transfer coefficient

```
[FoamVariables]
  [T_shadow]
    type = FoamVariableField
    foam_variable = 'T'
  []
  [e_shadow]
    type = FoamVariableField
    foam_variable = 'e'
  []
  [whf_shadow]
    type = FoamFunctionObject
    foam_variable = 'wallHeatFlux'
  []
[]
```

# Foam-based Postprocessors

- These are Postprocessors that are directly applied to the OpenFOAM mesh
  - Names mirror MOOSE counterparts
  - Variants for primitive variables and function objects
- Current postprocessors
  - FoamSideAdvectionFluxIntegral (for calculating the mass flow rate)
  - FoamSideAverageValue
  - FoamSideAverageFunctionObject
  - FoamSideIntegratedValue
  - FoamSideIntegratedFunctionObject

```
[Postprocessors]
  [m_dot_x1]
    type = FoamSideAdvectionFluxIntegral
    foam_scalar = 'rho'
    boundary = left
  []

  [t_avg]
    type = FoamSideAverageValue
    foam_variable = 'T'
    boundary = top
    execute_on = TIMESTEP_END
  []

  [heat_flux]
    type = FoamSideIntegratedFunctionObject
    function_object = 'wallHeatFlux'
    boundary = right
    component = x
  []
```

# Hippo: capabilities and usage

- Example: for CHT two MOOSE input files are used
  - Solid.i: defines solid heat conduction and transfers
  - Fluid.i: defines which variables are copied to the mirror and imposition of BCs
  - Right: snippet of input file for CHT

```
hippo_case
├── solid.i # Heat conduction MOOSE input file
├── fluid.i # OpenFOAM MOOSE input file
├── fluid-foam # OpenFOAM case directory
│   ├── 0
│   ├── constant
│   └── system
```

```
# fluid.i

...

[FoamVariables]
  [T_wall]
    type = FoamVariableField
    foam_variable = 'T'
    boundary = wall
  []
[]

[FoamBCs]
  [wall_heat_flux]
    type = FoamFixedGradient
    foam_variable = 'T'
    diffusivity_coefficient=kappa
    boundary = wall
  []
[]

...
```

# Future work

- Add support for volumetric coupling
  - Swap a mirror mesh for direct transfers from MOOSE to OpenFOAM
- Support for OpenCFD-ESI flavour of OpenFOAM.
- More demonstration cases
  - Flow boiling in a hypervapotron
  - HIVE